

TemaFinale26 – Sprint 1: Analisi del Problema

Scopo e goal dello Sprint 1

Questa fase fa **analisi del problema**: a partire dai requisiti già formalizzati ([TemaFinale26 – Analisi dei Requisiti](#)) si costruisce un **modello logico** della soluzione — *di che tipo* sono i componenti, *come* interagiscono e *quale* comportamento ci si attende — **senza** ancora scegliere i dettagli di realizzazione (codice qak concreto, classi, trasporto), che sono compito delle fasi di **Progetto e Development**.

Goal dello Sprint 1 (sottoinsieme del testo del committente). Analizzare e modellare il **core business**: "*the cargoservice uses the cargorobot to move the container from the IOPort to the slot5 (for marking the container) and then to the reserved slot*", con ritorno del cargorobot a HOME e sistema pronto per una nuova richiesta. Gli aspetti accessori (display come web-gui, LED, *Out of service*, barcode) sono rinviati a iterazioni successive (confermato dal committente).

Metodo (altitudine). I requisiti dicono **che cosa**. L'analisi del problema si chiede: *quali tipi di componente software* realizzano le entità del committente, *come* interagiscono e *quale* comportamento producono — tutto a livello **logico**. La scelta della tecnologia concreta, dei contratti sintattici e del trasporto è **Progetto**. Per ogni decisione, qui sotto, si indica se è un **requisito**, una scelta di **analisi** (motivata) o una scelta rinviata al **progetto**.

Di che tipo sono i componenti? (pojo / service / attore)

Prima di progettare occorre **analizzare la natura** di ogni entità del committente. Si usano tre categorie del metamodello:

- **POJO** — oggetto *passivo*, senza vita propria, invocato in modo **sincrono**; incapsula dati/stato e calcolo (nessun messaggio).
- **Service** — componente che *espone operazioni* e **risponde a richieste** (request/reply) di altri: è **reattivo**, guidato dalle richieste, ma non ha un piano proprio.
- **Attore (QActor)** — entità *autonoma* con stato, insieme **proattiva** (ha un proprio comportamento/piano) e **reattiva** (a messaggi ed eventi), che comunica **solo** per messaggi asincroni.

Entità del committente	Discussione (analisi)	Natura (conclusione)
cargoservice	deve <i>guidare</i> una sessione di carico in più passi (riserva slot, attesa container, IOPort→slot5→slot, HOME) e insieme <i>reagire</i> a pushbutton, sensor, marker e timeout. Un pojo o un service puro non bastano: non "guidano" un processo, rispondono soltanto.	attore (proattivo + reattivo)

cargorobot (DDR)	componente fornito dal committente, che raggiunge una posizione su comando e risponde con l'esito; <i>quale</i> software del DDR usare e' analizzato a parte (vedi sezione seguente).	service esterno
sensor dell'IOPort	fornisce la distanza D di un oggetto davanti all'IOPort. Il sonar e' una <i>parte</i> del sensor (il dispositivo di misura), non un componente a se'. Come software e' una <i>sorgente di notifiche</i> sulla presenza del container/guasto: cosa sia esattamente e come fornisca il dato resta da chiarire col committente .	sorgente di notifiche (service/attore) — <i>aperto</i>
IOPort	non e' un unico componente: il pushbutton e' la sorgente di una <i>richiesta</i> (interazione request/reply: e' un requisito); il display e' un <i>canale di output</i> . Un analista puo' sostenere che il display faccia parte della web-gui , non del cargoservice.	pushbutton: sorgente di richieste · display: output (forse web-gui)
marker (slot5)	etichetta il container e segnala il completamento: al cargoservice serve solo il segnale "fatto".	service/attore che notifica un evento
LED	riflette lo stato <i>engaged</i> del sistema: e' un indicatore di stato , non un attuatore (non agisce sull'ambiente).	indicatore di stato (output)
Hold (stiva)	stato del dominio (slot, occupazione, posizioni dalla mappa): nessuna vita propria, invocato in modo sincrono dal cargoservice.	POJO

Quante di queste parti siano attori/servizi autonomi e' una *conclusione* dell'analisi, non un dato di partenza; per **sensor** e **display** restano domande aperte da chiarire col committente.

Quale software per il cargorobot, e perche'

Il committente **non** fornisce un solo robot, ma un **insieme di componenti** (POJO e servizi) per il DDR. Quale usare per realizzare il cargorobot e' oggetto di **analisi** (non un semplice rinvio al progetto):

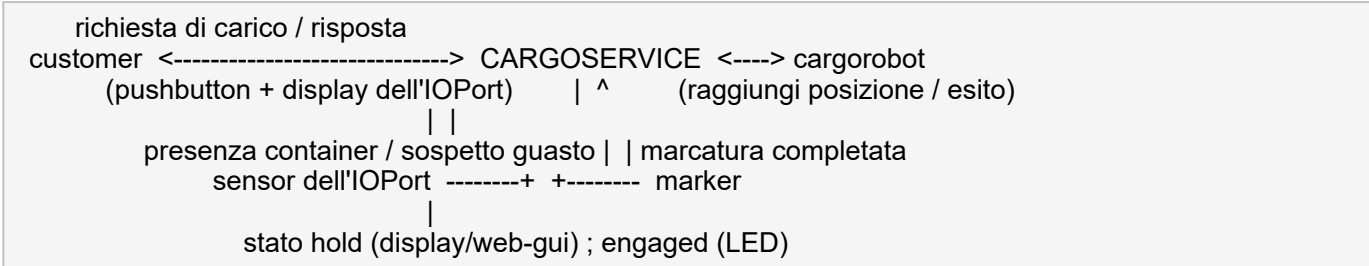
Componente del DDR	Cosa offre	Adeguatezza al problema
RobotObj26 (POJO)	comandi elementari sincroni (move l/r/a, step)	troppo basso livello: il cargoservice dovrebbe gestire la navigazione mossa-per-mossa
RobotService26 (service)	il robot esposto come servizio (move/cmd/step + eventi)	astrae il trasporto, ma resta al livello delle <i>mosse</i> elementari
robotsmart (service)	moverobot(TARGETX, TARGETY) orientato all'obiettivo (pianificazione A* + mappa)	adeguato : lavora a livello di <i>posizione-obiettivo</i>

Il problema chiede di **"raggiungere una posizione"** (IOPort, slot5, slot riservato, HOME): e' esattamente l'astrazione offerta da moverobot(X,Y). Si analizza quindi come candidato motivato

robotsmart, usato come **service esterno** di cui il cargoservice e' client (lo stesso pattern con cui boundaryworker usa robotservice). Cosi' il cargoservice resta al livello del *dominio* (posizioni) e non delle mosse del robot. La conferma della versione esatta e' una rifinitura di **Progetto**; la **motivazione** della scelta e' analisi.

Modello delle interazioni

Il modello delle interazioni descrive **chi interagisce con chi, perche', e di che natura** e' ciascun flusso (non e' una semplice lista di messaggi). Il cargoservice e' il centro:



Flusso	Natura	Altitudine	Motivazione
pushbutton → cargoservice	request / reply	requisito	il committente: "send a request ... show the answer to the request"
sensor → cargoservice	notifica (presenza/guasto)	analisi	sorgente asincrona senza risposta → si sceglie l' <i>evento</i> (alternativa: dispatch); il sonar e' parte del sensor
marker → cargoservice	notifica "fatto"	analisi	basta il segnale di completamento → evento
cargoservice → cargorobot	request / reply	contratto (vedi sopra)	serve sapere <i>quando</i> il robot ha raggiunto la posizione per procedere
cargoservice → display / LED	output di stato	progetto / aperto	display forse parte della web-gui; LED indicatore di stato

Il **trasporto** (TCP / CoAP / MQTT) **non** e' deciso qui: e' Progetto. A livello di analisi contano i *ruoli*, la *natura* dei flussi e l'*informazione* scambiata.

Formalizzazione (qak) delle interazioni di base

Come indicato dal docente, l'interazione che e' gia' un **requisito** — la request/reply del pushbutton — si puo' **formalizzare subito in qak**, a livello di *interfaccia dei messaggi* (non di realizzazione). Allo stesso modo si riporta il **contratto** (esterno) offerto da robotsmart per il cargorobot:

```

//---- pushbutton -> cargoservice (REQUISITO)
Request loadrequest : loadrequest()
Reply answer       : answer(RES) for loadrequest
                   RESP = reserved(SLOT) | retrylater | reject

//---- cargoservice -> cargorobot (contratto di robotsmart, scelta motivata sopra)
  
```

```
Request moverobot : moverobot(TARGETX, TARGETY, STEPTIME)
Reply moverobotdone : moverobotok(ARG) for moverobot
Reply moverobotfailed: moverobotfailed(PLANDONE, PLANTODO) for moverobot
```

Questa e' la **formalizzazione delle interfacce**, non il comportamento interno ne' la realizzazione: la traduzione in una **FSM qak** concreta (stati, transizioni, `withobj`, codice) e' compito di **Progetto/Development**. Le user stories dello Sprint 0 (US1..US10) prefigurano gia' il primo *test plan*.

Comportamento atteso (livello di problema)

A livello di problema il cargoservice attraversa una **sessione di carico** fatta di fasi logiche (non si fissa qui la FSM concreta):

```
disponibile (Service working)
  | richiesta valida (non pieno, IOPort libero)
  v
engaged -- riserva uno slot, risposta reserved(slot), LED on
  | container presente all'IOPort entro ~30s (altrimenti: disengage, slot liberato)
  v
trasporto -- il cargorobot: IOPort -> slot5 (marcatura) -> slot riservato
  |
  v
ritorno -- il cargorobot torna a HOME, slot confermato occupato
  v
disponibile (pronto per una nuova richiesta)
```

Vincoli dai requisiti: **una sola richiesta per volta** (le altre ricevono retrylater); stiva piena → reject; IOPort occupato o sistema Out of service → retrylater. La gestione dell'*Out of service* (sospetto guasto del sensor) e del barcode e' rinviata a un'iterazione successiva (confermato dal committente).

La **traduzione** di queste fasi in una macchina a stati qak concreta (con State/Transition, `withobj` e codice) appartiene alla fase di **Progetto/Development**: qui si definisce *quale* comportamento, non *come* scriverlo.

Core business e strategia incrementale

Iterazione	Cosa realizza
Prototipo 1 (core)	flusso nominale: richiesta valida → riserva slot → container all'IOPort → IOPort→slot5→slot riservato → HOME. <i>Out of service</i> e barcode ignorati (confermato dal committente).
Iterazione 2	rifiuti (hold pieno → reject; IOPort occupato → retrylater) e timeout dei 30s.
Iterazione 3	<i>Out of service</i> (sospetto guasto del sensor) e ripristino; display come web-gui .

Decisioni e domande aperte

Tema	Altitudine	Stato
container considerato spostato quando il robot raggiunge la posizione (nessun attuatore di presa nel DDR)	analisi	confermato dal committente
cargorobot realizzato da robotsmart (service a posizione-obiettivo)	analisi (motivata)	versione esatta → Progetto
che tipo di componente sia il sensor e come fornisca il dato	analisi	aperta (committente)
display : parte della web-gui o del service?	analisi	aperta (committente)
tipo di interazione del sensor/marker (evento vs dispatch)	analisi	scelto evento, motivato
trasporto (MQTT / TCP / CoAP) e contratti sintattici	progetto	rinvio

Piano di test (livello di problema)

Il cargoservice mantiene lo **stato della stiva**, che evolve in conseguenza delle azioni: il test e' **automatizzabile** confrontando lo stato finale con quello atteso (PASS/FAIL). Si definisce qui *che cosa* verificare; l'implementazione del test e' Development.

1. **Carico nominale (test significativo per la demo)**: stiva con uno slot libero, richiesta valida, container all'IOPort entro 30s. Asserzioni finali:

```
slot riservato .occupied == true
posizione cargorobot    == HOME
stato cargoservice      == disponibile
```

2. **Stiva piena**: slot1..4 occupati → risposta **reject**, stato della stiva **invariato**.
3. **Retrylater**: IOPort occupato o sistema Out of service → risposta **retrylater**, stiva invariata.
4. **Timeout**: richiesta accettata ma container non arriva entro 30s → disengage, slot riservato di nuovo libero.

Il caso 1 e' il **Test Plan significativo** richiesto per la demo: copre l'intero ciclo della sessione di carico e termina con asserzioni PASS/FAIL.



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer